Milestone 3 - INDIVIDUAL DESIGN PLAN
CitySafe – Smart Navigation System Prototype (v1)

Script Squad – Daniel John Varghese

I. Prototype Overview

(i) Features

The objective of Prototype v1 is to establish a functional **"Safety Loop"** that validates CitySafe's core value within the three-week development window. Rather than implementing the full set of features defined in Milestone 2, this prototype prioritises life-safety functionality, community resilience, and accessibility, while deferring engagement and extended integration features.

The MVP validates the following core workflow:

Hazard Awareness → Community Contribution → Safety Alerts → Assistance Response

Feature selection is guided by:

- User safety impact
- Alignment with UN Sustainable Development Goal 11
- Technical feasibility within the allocated timeline
- Architectural stability for future expansion

Feature (Milestone 2 ID)	Priority	Justification
F-1 Interactive Safety Map	High	Enables real-time hazard visualisation and supports safe navigation decisions
F-4 User Hazard Reporting	High	Facilitates community-driven data contribution
F-2 Safety Alerts	High	Provides proactive risk notification via geofencing
F-6 Call for Help (SOS)	High	Enables peer-supported urban resilience
F-7 Accessibility Display Settings	High	Ensures inclusive access to safety-critical information
F-8 User Accounts	Medium	Enables accountability and structured report ownership
F-3 Safe Navigation	Partial	Implemented using overlay-based hazard avoidance

The combination of F-1, F-4, and F-2 establishes the system's core safety layer by enabling real-time hazard visibility and proactive alerting. The simplified verification mechanism ensures reliability without implementing complex credibility scoring.

Hazard reports follow a structured lifecycle to ensure reliability of community-submitted data:

Submitted → Under Review → Verified / Rejected → Resolved

This lifecycle allows the system to validate reports before generating safety alerts, improving the accuracy of information displayed on the safety map.

F-6 operationalises community resilience through a structured SOS lifecycle:

Pending → Accepted → In Progress → Resolved

When a citizen sends an SOS request, the system broadcasts the request to nearby responders. Once a responder accepts the request, the system updates the request status until the emergency assistance is completed.

Accessibility (F-7) is implemented in accordance with WCAG 2.1 principles, ensuring that safety-critical information remains perceivable for users with visual impairments.

The following features are intentionally deferred to preserve scope control:

- Points and reward system (F-10)
- Tourist recommendations (F-12)
- Car system integration (F-13)
- Advanced credibility scoring (F-11 extension)

Prototype v1 therefore validates CitySafe's core safety objectives while remaining technically achievable within the project timeline.

II. Technology Stack

To implement Prototype v1 within the three-week timeframe, CitySafe adopts a modular client–server architecture that prioritises development efficiency, scalability, accessibility compliance, and geospatial processing capability.

(i) Recommended Stack

Stack	Technological Component	Purpose
Frontend	React Native (JavaScript/TypeScript)	Cross-platform mobile application
Backend	Node.js + Express	REST API and business logic layer
Database	PostgreSQL + PostGIS	Relational storage with spatial queries
Maps Integration	Google Maps API	Map rendering and route overlay
Notifications	Firebase Cloud Messaging (FCM)	Push alerts for hazards and SOS

(ii) Justification

Development Feasibility

React Native enables simultaneous iOS and Android deployment from a single codebase, reducing frontend development time within the limited project window. This supports rapid prototyping while maintaining accessibility compliance (WCAG 2.1). Node.js with Express provides a lightweight RESTful backend suitable for I/O-intensive operations such as hazard reporting and SOS broadcasting. Its non-blocking architecture supports concurrent user activity without requiring complex infrastructure.

Requirement Alignment

PostgreSQL extended with PostGIS was selected due to the system's reliance on geospatial queries. Core features such as:

- Identifying nearby hazards (F-1)
- Triggering proximity-based alerts (F-2)
- Broadcasting SOS requests to nearby volunteers (F-6)

require efficient spatial indexing and distance calculations, which PostGIS natively supports.

Google Maps API is integrated for reliable map rendering and route visualisation, allowing partial implementation of safe navigation (F-3) without developing a custom routing engine. Firebase Cloud Messaging (FCM) supports geofenced hazard alerts and SOS notifications, eliminating the need for custom real-time socket infrastructure in Prototype v1.

(iii)Scope Boundaries

To preserve architectural stability within the development timeframe:

- Navigation is implemented as hazard-overlay guidance rather than dynamic route optimisation.
- Verification uses manual confirmation rather than algorithmic credibility scoring.
- WebSocket-based live streaming is deferred in favour of push notifications.

These constraints ensure that Prototype v1 validates core safety functionality without overextending technical scope.

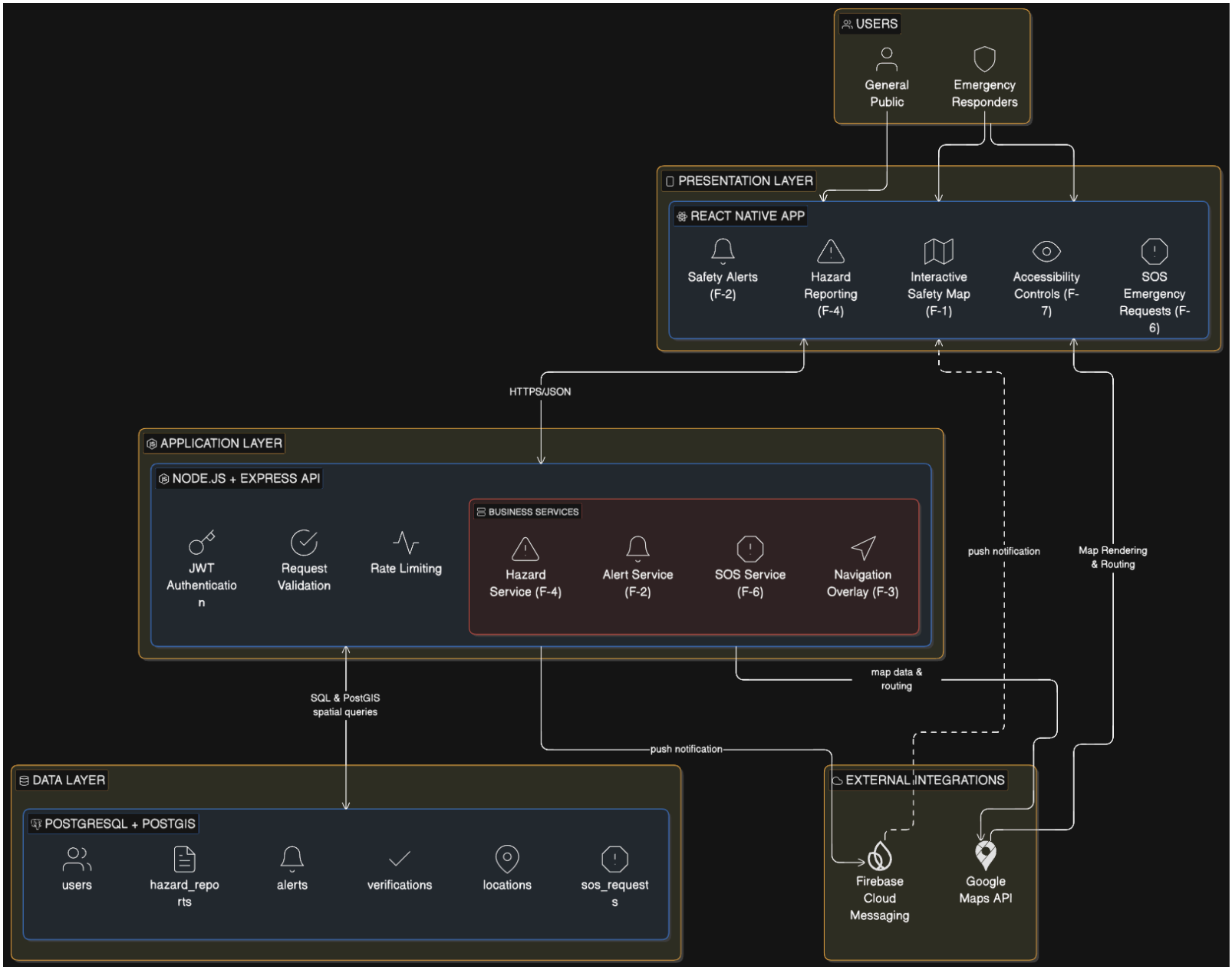
III. System Architecture Model:

CitySafe follows a layered (three-tier) client–server architecture designed to support modular development and clear separation of responsibilities.

Layer	Description
Presentation Layer (React Native Mobile Application)	Handles user interaction, including the Safety Map (F-1), Hazard Reporting (F-4), Safety Alerts (F-2), SOS requests (F-6), and Accessibility Controls (F-7). The mobile client communicates with the backend via secure HTTPS/JSON requests.
Application Layer (Node.js + Express REST API)	Manages authentication (JWT), request validation, and core business logic. This layer processes hazard submissions, verification updates, proximity-based alert generation, and SOS broadcast workflows.
Data Layer (PostgreSQL + PostGIS)	Provides persistent relational storage with spatial indexing to support geospatial queries such as identifying nearby hazards and volunteers.

This layered separation ensures maintainability, enables parallel team development, and isolates safety-critical processing from presentation components while remaining feasible within the three-week timeframe.

System Architecture Model Diagram (Figure 1)



IV. High-Level Feature Models

(i) Use Case Diagram (Figure 2)

Category	Details
Actors	• Citizen (general public user), • Responder (volunteer or nearby helper)
Core Use Cases	• View Safety Map • Report Hazard • Receive Safety Alerts • Request SOS Help
System Interactions	Hazard reports trigger verification and map updates, while SOS requests broadcast alerts to nearby responders

(ii) UML Class Diagram (Figure 3)

Category	Details
Key Classes	User, Citizen, Responder, HazardReport, Location, Alert, EmergencyRequest, Route
Inheritance	Citizen and Responder inherit from the base User class
Key Relationships	Citizens submit hazard reports, responders handle emergency requests, and hazard/route data reference geographic locations
Purpose	The class diagram models the object structure of the CitySafe system and how core entities interact within the application

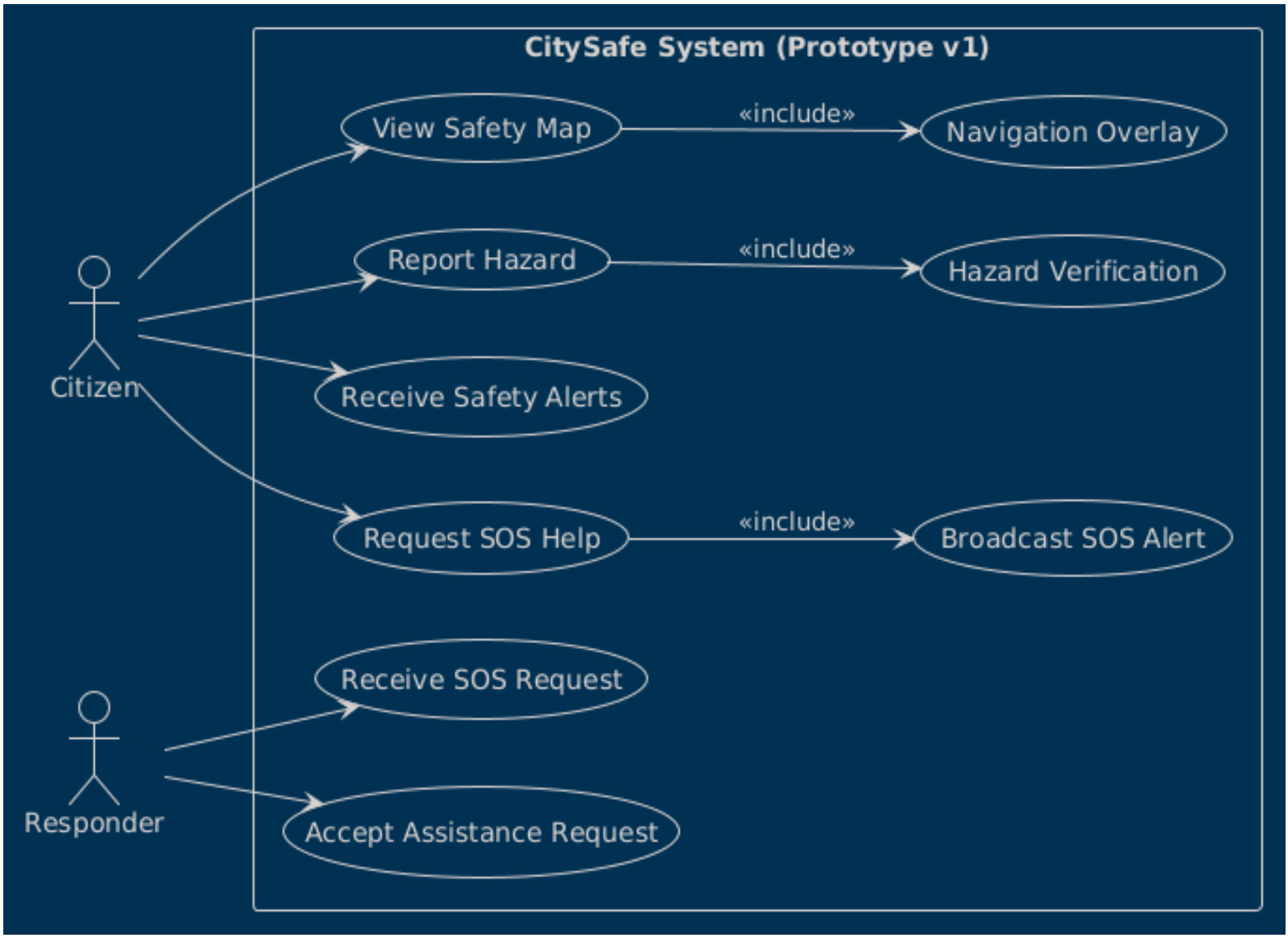
(iii) Sequence Diagram (Figure 4)

Flow	Description
Emergency Assistance Workflow	The sequence diagram illustrates how a citizen interacts with the CitySafe system during an emergency. A citizen can view the safety map and submit hazard reports through the mobile application. The backend system stores hazard data in the database and broadcasts safety alerts to nearby users. Safety alerts are generated when a hazard report is verified and users are located within a defined geographic radius of the hazard location. If a citizen requests SOS assistance, the system creates an emergency request and notifies nearby responders. A responder can accept the request, after which the system updates the request status and confirms assistance to the citizen.

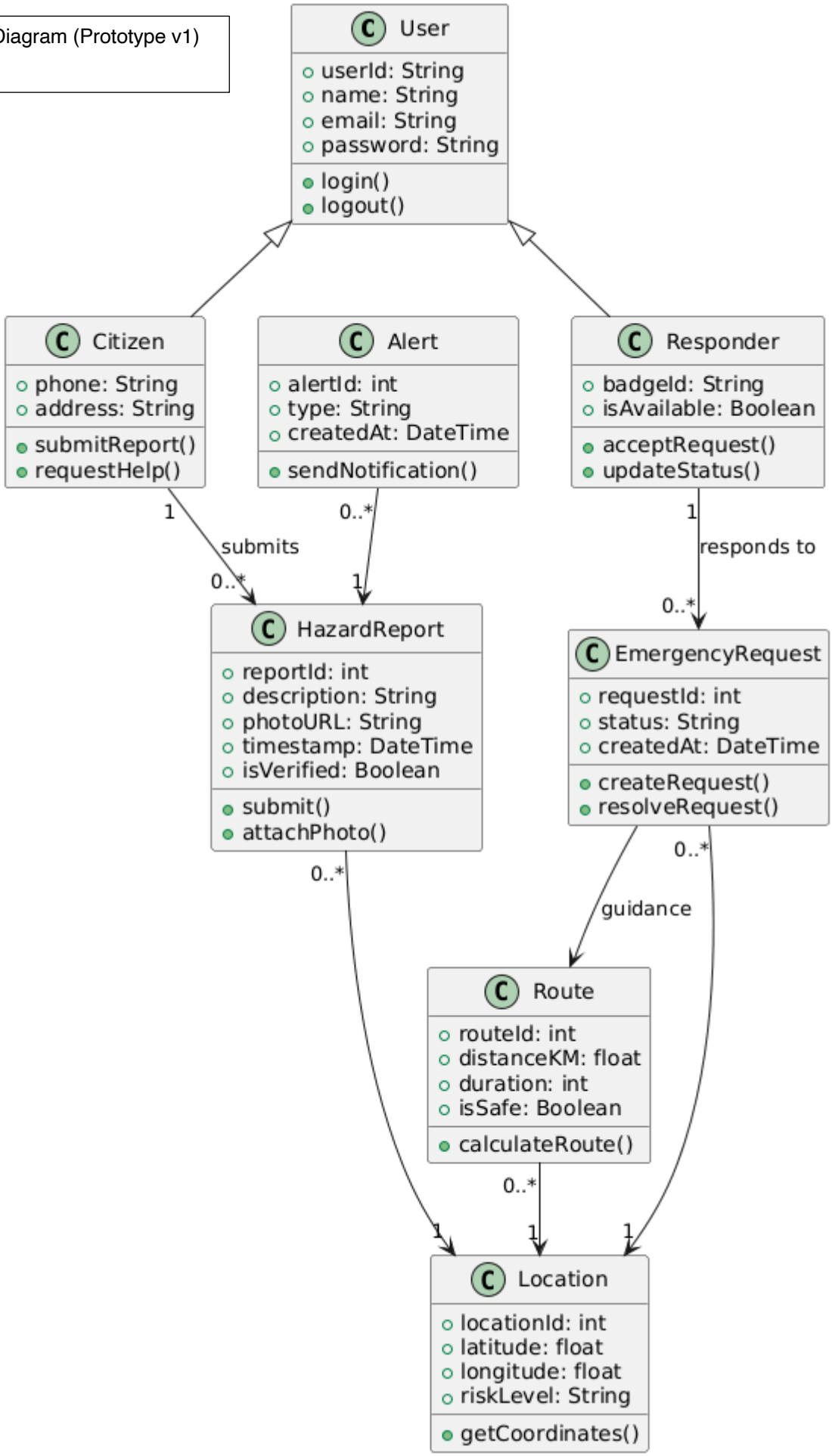
(iv) ER Diagram (Figure 5)

Category	Details
Core Entities	Users, Citizens, Responders, HazardReports, EmergencyRequests, Alerts, Locations, Routes
Key Relationships	Citizens submit hazard reports, hazard reports generate alerts, citizens create emergency requests, responders respond to requests
Location Data	Hazard reports and emergency requests reference geographic locations
Navigation Data	Routes connect locations and provide safe navigation information
Purpose	The ER diagram models the database structure required to store users, hazards, alerts, and emergency assistance data within the CitySafe system

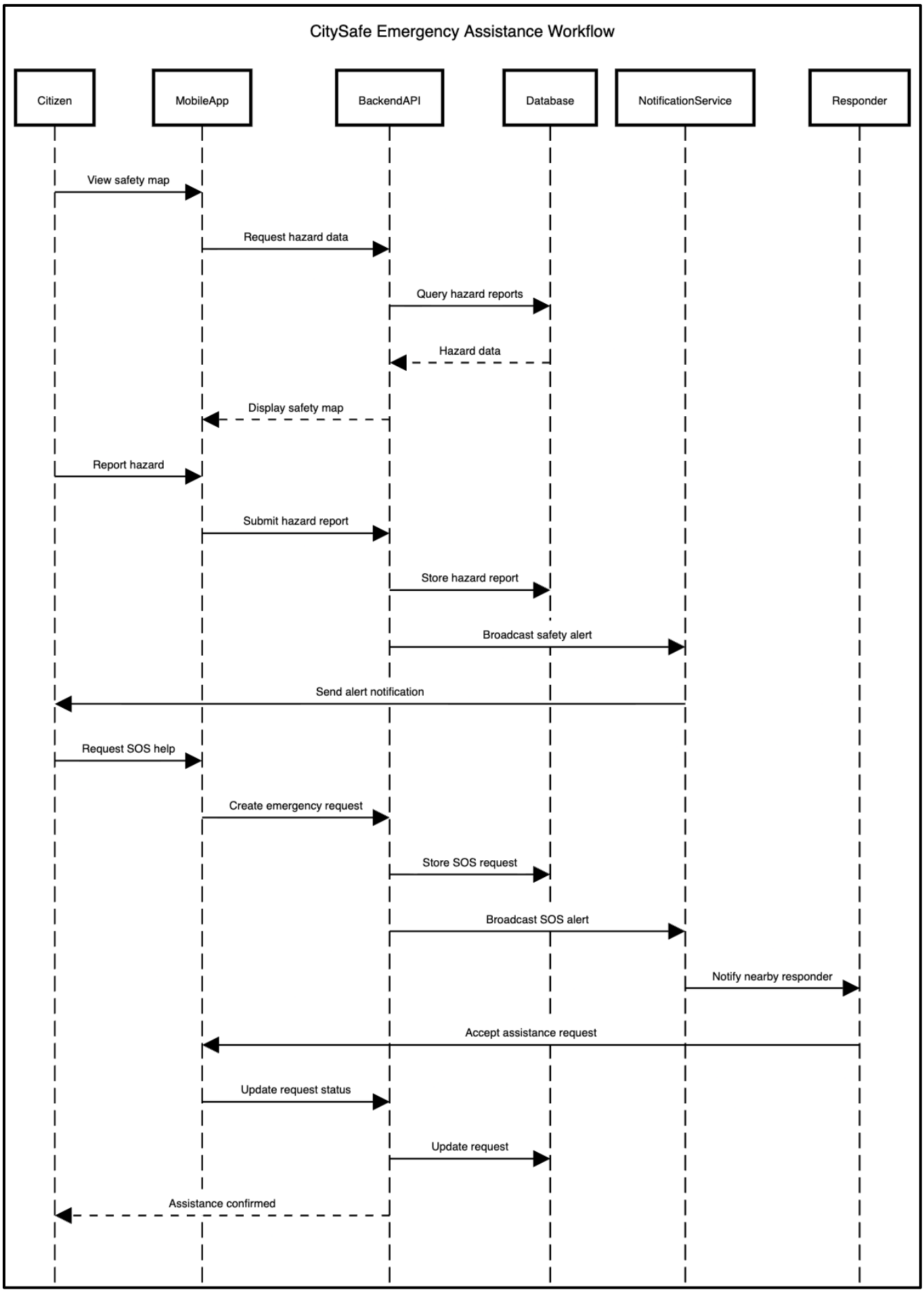
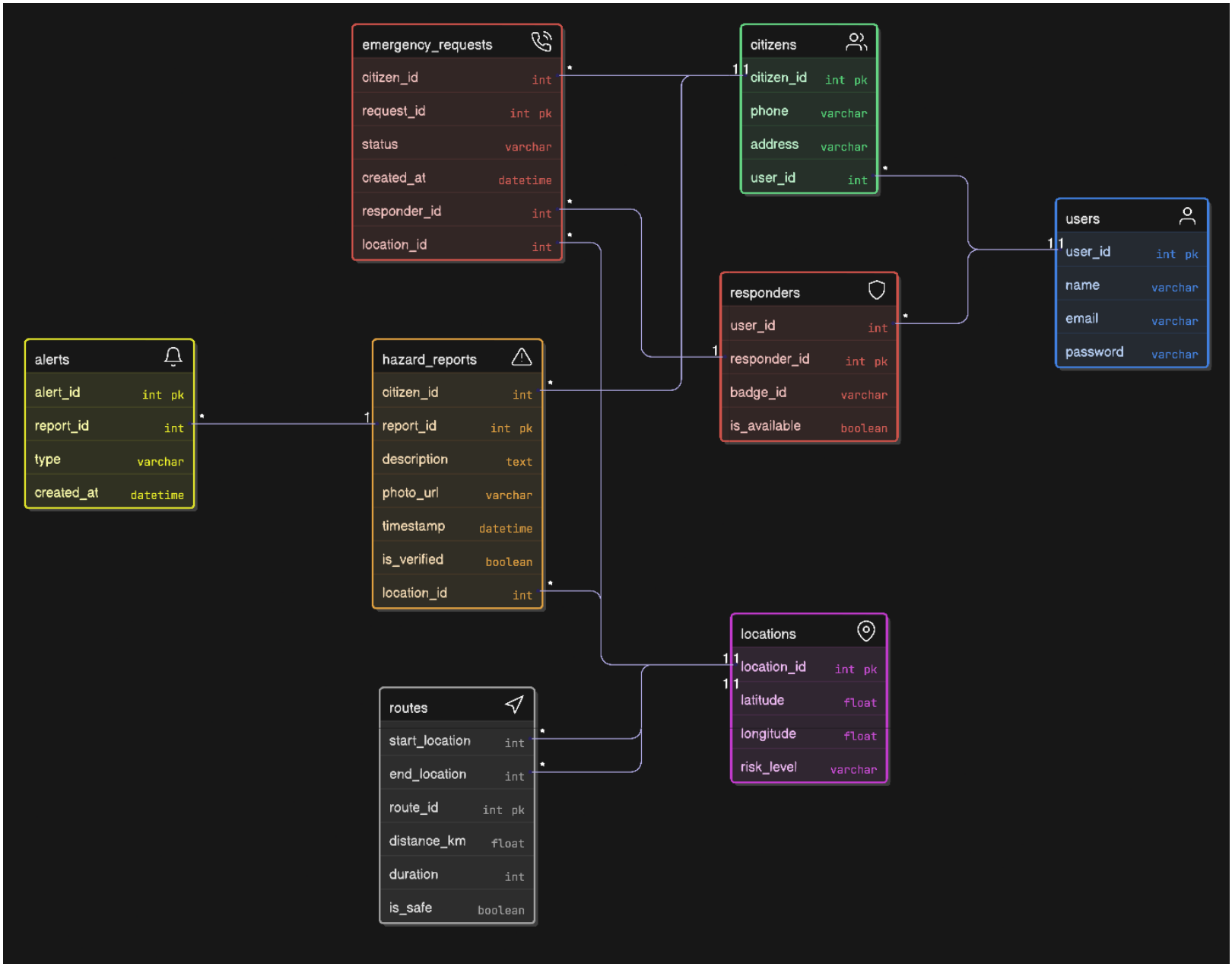
Use-Case Diagram (Prototype v1)



UML Class Diagram (Prototype v1)



Sequence Diagram (Prototype v1)



ER Diagram (Prototype v1)